

Task 1

Task 001

```
import java.util.LinkedList;

public class Task001_DS_LinkedList {
    public static void main(String[] args) {
        LinkedList<String> fruits = new LinkedList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.addFirst("Orange");
        fruits.addLast("Grapes");

        System.out.println("First Element: " + fruits.getFirst());
        System.out.println("Last Element: " + fruits.getLast());

        fruits.removeFirst();
        fruits.removeLast();

        for (String fruit : fruits) {
            System.out.println(fruit);
        }
    }
}
```

```

import java.util.LinkedList;

public class Task001_DS_LinkedList {
    public static void main(String[] args) {
        LinkedList<String> fruits = new LinkedList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.addFirst(e: "Orange");
        fruits.addLast(e: "Grapes");

        System.out.println("First Element: " + fruits.getFirst());
        System.out.println("Last Element: " + fruits.getLast());

        fruits.removeFirst();
        fruits.removeLast();

        for (String fruit : fruits) {
            System.out.println(fruit);
        }
    }
}

```

```

First Element: Orange
Last Element: Grapes
Apple
Banana

```

Home task

Advantages of Linked Lists:

1. Dynamic Size

Linked lists can grow or shrink in size as needed. You don't need to define the size in advance, unlike arrays.

2. **Efficient Insertion and Deletion**
Inserting or deleting elements from the beginning or middle is fast because you only need to change pointers. There's no need to shift elements like in an array.
3. **No Memory Waste**
Memory is allocated only when a new node is created. So, you only use memory when needed.
4. **Good for Implementing Other Data Structures**
Linked lists are the foundation for other data structures like stacks, queues, and graphs.

Disadvantages of Linked Lists:

1. **No Direct Access (Slow Lookup)**
You cannot access elements directly by index. To get to an element, you must start from the head and go node by node.
2. **Extra Memory Usage**
Each node in a linked list stores not just data, but also a pointer to the next node. This adds memory overhead.
3. **More Complex Code**
Inserting, deleting, or reversing nodes requires more code and careful pointer handling compared to arrays.
4. **Poor Cache Performance**
Since elements are not stored in continuous memory blocks, they are less friendly to CPU caching, which can make access slower compared to arrays.

Application of Linked list

Dynamic Memory Management

Linked lists are used in memory management systems (like heap allocation) where memory is allocated and deallocated at runtime. They help in maintaining free memory blocks (free lists).

2. Implementation of Data Structures

Linked lists are used to build more complex data structures such as:

- Stacks
 - Queues
 - Hash Tables (for chaining in collision handling)
 - Graphs (Adjacency Lists)
-

3. Manipulating Polynomials or Big Numbers

In mathematical computations involving polynomials or large numbers, each term or digit can be stored in a linked list node, making it easy to insert, delete, or update dynamically.

4. Operating Systems

- Job scheduling and process management use linked lists to manage processes and tasks.
 - The file allocation table (FAT) in older file systems can be implemented using a linked list structure.
-

5. Web Browsers (Back/Forward Navigation)

Doubly linked lists are used to implement back and forward navigation in web browsers. Each page is a node with links to previous and next pages.

Task 002

Try to create a node and add a value to it..

- ☒ ~~Create a node~~
14 pax done with .. others .. still working .. @12.09
16 pax done with –@12.16



☐ Try to add element at the end of the list

☐ Add the element

11 Pax done out of 19 @ 12.25

☐ Remove the node

☐ Display all the elements of the node

☐ Find size of the linked list

☐ Index out of bounds

11.35 to 11.40

```
class Node<T> {
    T data;
    Node<T> next;

    public Node(T data) {
        this.data = data;
        this.next = null;
    }
}

public class CustomLinkedList<T> {
    private Node<T> head;
    private int size = 0;

    public void add(T data) {
        Node<T> newNode = new Node<>(data);
        if (head == null) {
            head = newNode;
        } else {
            Node<T> current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
        size++;
    }

    public void addFirst(T data) {
        Node<T> newNode = new Node<>(data);
        newNode.next = head;
        head = newNode;
    }
}
```

```

        size++;
    }

    public T removeFirst() {
        if (head == null) {
            throw new NoSuchElementException("List is empty");
        }
        T removedData = head.data;
        head = head.next;
        size--;
        return removedData;
    }

    public T get(int index) {
        checkBounds(index);
        Node<T> current = head;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
        return current.data;
    }

    public int size() {
        return size;
    }

    private void checkBounds(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Index out of bounds");
        }
    }
}

public class Task002_DS_CustomLinkedList {
    public static void main(String[] args) {
        CustomLinkedList<String> liobj = new CustomLinkedList<>();
        liobj.add("Anitha");
        liobj.add("Verma");
        liobj.addFirst("Jack");

        System.out.println("First Element: " + liobj.get(0));
        System.out.println("Size: " + liobj.size());

        liobj.removeFirst();

        System.out.println("First Element after removal: " + liobj.get(0));
        System.out.println("Size after removal: " + liobj.size());
    }
}

```

```
}  
}
```

```
public class Task02_day14<V> {  
      
    static class Node<T> { 14 usages  
        T data; 4 usages  
        Node<T> nextNode; 12 usages  
  
        public Node(T data) { 3 usages  
            this.data = data;  
            this.nextNode = null; // Set nextNode to null initially  
        }  
    }  
  
    private Node<V> headNode; // Head of the list 12 usages  
    private int size; // Size of the list 10 usages  
  
    // Check if the list is empty  
    public boolean isEmpty() { 6 usages  
        return headNode == null;  
    }  
  
    // Get the size of the list  
    public int size() { 2 usages  
        return size;  
    }  
  
    // Add element at the end of the list  
    public void add(V value) { 4 usages
```

```

public class Task02_Day14<V> {
    // Add element at the end of the list
    public void add(V value) { 4 usages
        Node<V> toAddNode = new Node<V>(value);
        if (isEmpty()) {
            headNode = toAddNode;
        } else {
            Node<V> current = headNode;
            while (current.nextNode != null) {
                current = current.nextNode;
            }
            current.nextNode = toAddNode;
        }
        size++;
    }

    // Add element at the beginning of the list
    public void addFirst(V value) { 3 usages
        Node<V> toAddNode = new Node<V>(value);
        if (isEmpty()) {
            headNode = toAddNode;
        } else {
            toAddNode.nextNode = headNode;
            headNode = toAddNode;
        }
        size++;
    }
}

```



```

        size++;
    }

    // Remove the first element of the list
    public V removeFirst() { 1 usage
        if (isEmpty()) {
            return null; // Return null if list is empty
        } else {
            Node<V> oldHeadNode = headNode;
            headNode = headNode.nextNode;
            size--;
            return oldHeadNode.data;
        }
    }

    // Get the element at a specific index
    public V getElementAt(int index) { 2 usages
        if (isEmpty()) {
            throw new IllegalArgumentException("Linked List is Empty");
        }
        if (index >= size || index < 0) {
            throw new IllegalArgumentException("Out Of Bounds, index starts at 0 and ends at " +
            size - 1);
        }
        Node<V> current = headNode;

```

```
// Get the element at a specific index
public V getElementAt(int index) { 2 usages
    if (isEmpty()) {
        throw new IllegalArgumentException("Linked List is Empty");
    }
    if (index >= size || index < 0) {
        throw new IllegalArgumentException("Out Of Bounds, index starts at 0 and ends at " +
        );
    }
    Node<V> current = headNode;
    for (int i = 0; i < index; i++) {
        current = current.nextNode;
    }
    return current.data;
}

// Add element at a specific index
public boolean addAt(int index, V value) { 1 usage
    if (isEmpty() && index == 0) {
        addFirst(value);
        return true;
    }
    if (index < 0 || index > size) {
        throw new IllegalArgumentException("Out Of Bounds, index starts at 0 and ends at " +
        );
    }
    if (index == 0) {
        addFirst(value);
    }
}
```

89:26 CRLF UTF-8 4 spaces

```
if (index < 0 || index > size) {
    throw new IllegalArgumentException("Out Of Bounds, index starts at 0 and ends at " +
}
if (index == 0) {
    addFirst(value);
    return true;
}
if (index == size) {
    add(value);
    return true;
}

Node<V> current = headNode;
Node<V> prev = null;
Node<V> newNode = new Node<V>(value);

for (int i = 0; i < index; i++) {
    prev = current;
    current = current.nextNode;
}
prev.nextNode = newNode;
newNode.nextNode = current;
size++;
return true;
```

```

        prev.nextNode = newNode;
        newNode.nextNode = current;
        size++;
        return true;
    }

    // Override the toString method to display the list elements
    @Override
    public String toString() {
        if (isEmpty()) {
            return "[]";
        } else {
            StringBuilder sb = new StringBuilder();
            sb.append("[ ");
            Node<V> current = headNode;
            while (current != null) {
                sb.append(current.data);
                if (current.nextNode != null) {
                    sb.append(" -> ");
                }
                current = current.nextNode;
            }
            sb.append(" ]");
            return sb.toString();
        }
    }

```

```

public String toString() {
    sb.append(" ");
    return sb.toString();
}

public static void main(String[] args) {
    // Create a new singly linked list of Integer type
    Task02_day14<Integer> nums = new Task02_day14<>();

    // Add elements to the linked list
    nums.add(10);
    nums.add(20);
    nums.addFirst(value: 44);
    nums.add(40);
    nums.addAt(index: 3, value: 66); // Add at index 3

    // Print the linked list and its size
    System.out.println(nums); // Output: [ 44 -> 10 -> 20 -> 66 -> 40 ]
    System.out.println("Size of the list: " + nums.size()); // Output: 5

    // Remove the first element and print the list
    nums.removeFirst();
    System.out.println("After removing the first element: " + nums); // Output: [ 10
    System.out.println("Size after removal: " + nums.size()); // Output: 4

```

```

public static void main(String[] args) {
    // Create a new singly linked list of Integer type
    Task02_day14<Integer> nums = new Task02_day14<>();

    // Add elements to the linked list
    nums.add(10);
    nums.add(20);
    nums.addFirst(value: 44);
    nums.add(40);
    nums.addAt(index: 3, value: 66); // Add at index 3

    // Print the linked list and its size
    System.out.println(nums); // Output: [ 44 -> 10 -> 20 -> 66 -> 40 ]
    System.out.println("Size of the list: " + nums.size()); // Output: 5

    // Remove the first element and print the list
    nums.removeFirst();
    System.out.println("After removing the first element: " + nums); // Output: [ 10 -> 20 -> 66 -> 40 ]
    System.out.println("Size after removal: " + nums.size()); // Output: 4

    // Get element at a specific index
    try {
        System.out.println("Element at index 2: " + nums.getElementAt(index: 2)); // Output: 20
    } catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());
    }
}

```

```

// Remove the first element and print the list
nums.removeFirst();
System.out.println("After removing the first element: " + nums); // Output: [ 10
System.out.println("Size after removal: " + nums.size()); // Output: 4

// Get element at a specific index
try {
    System.out.println("Element at index 2: " + nums.elementAt(index: 2)); //
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
}

// Trying to get an out-of-bound index (e.g., 5, which is greater than the current
try {
    System.out.println("Element at index 5: " + nums.elementAt(index: 5)); //
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage()); // Output: Out Of Bounds, index starts a
}
}
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\P
[ 44 -> 10 -> 20 -> 66 -> 40 ]
Size of the list: 5
After removing the first element: [ 10 -> 20 -> 66 -> 40 ]
Size after removal: 4
Element at index 2: 66
Out Of Bounds, index starts at 0 and ends at 3

```

```

Size after removal: 4
Element at index 2: 66
Out Of Bounds, index starts at 0 and ends at 3

Process finished with exit code 0

```

Task 3

List down the methods of linked lists..

remove(T data)

removeFirst()

removeLast()

removeAt(int index)

get(int index)

set(int index, T data)

contains(T data)

indexOf(T data)

lastIndexOf(T data)

display()

getSize()

isEmpty()

clear()

reverse()

sort()

Task 4:

What are the operations of data structures.. I liner

14.46 to 14.50

Traversing

Insertion

Deletion

Searching

Sorting

Traversing – Accessing each element of a data structure one by one.

Insertion – Adding a new element to the data structure.

Deletion – Removing an existing element from the data structure.

Searching – Finding the location or existence of a specific element.

Sorting – Arranging elements in a particular order (ascending or descending).

Task 5:

Wap to create a linked list add 5 elements to it and replace 3 rd element with different value..

```
public class LinkedListExample {  
  
    // Step 1: Create the Node class  
    static class Node { 7 usages  
        int data;    // Data to store 3 usages  
        Node next;  // Reference to the next node 6 usages  
  
        // Constructor to create a new node  
        public Node(int data) { 1 usage  
            this.data = data;  
            this.next = null; // Initially, the next node is null  
        }  
    }  
  
    // Step 2: Create the LinkedList class  
    static class LinkedList { 2 usages  
        Node head; // Head node of the list 8 usages  
        int size;  // Size of the linked list 4 usages  
  
        // Constructor to initialize an empty list  
        public LinkedList() { 1 usage  
            head = null;  
            size = 0;  
        }  
  
        // Method to check if the list is empty  
        public boolean isEmpty() { 1 usage
```

Activat

107.1 - 06/5/2020

```
// Method to check if the list is empty
public boolean isEmpty() { no usages
    return head == null;
}

// Method to get the size of the list
public int size() { no usages
    return size;
}

// Step 3: Add a new element at the end of the list
public void add(int value) { 5 usages
    Node newNode = new Node(value); // Create a new node
    if (head == null) {
        head = newNode; // If the list is empty, make the new node the head
    } else {
        Node current = head;
        // Traverse to the last node
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode; // Add the new node at the end
    }
    size++; // Increase the size of the list
}
```

Activate V

48:1 C:\Eto 83712c

```

        current.next = newNode, // Add the new node at the end
    }
    size++; // Increase the size of the list
}

// Step 4: Replace the element at a specific index
public void replaceElementAt(int index, int newValue) { 1 usage
    if (index < 0 || index >= size) {
        System.out.println("Index out of bounds");
        return;
    }

    Node current = head;
    for (int i = 0; i < index; i++) {
        current = current.next; // Traverse to the node at the given index
    }

    current.data = newValue; // Replace the data at the specified index
}

// Step 5: Print the entire linked list
public void printList() { 2 usages
    if (head == null) {
        System.out.println("The list is empty");
    }
}

```

Activate

69:1 C# to Java

```
public void printList() { // 2 usages
    System.out.println("The list is empty ");
    return;
}

Node current = head;
// Traverse and print each node's data
while (current != null) {
    System.out.print(current.data + " -> ");
    current = current.next;
}
System.out.println("NULL"); // End of the list
}
```

```
public static void main(String[] args) {
    // Step 6: Create a new LinkedList object
    LinkedList list = new LinkedList();

    // Step 7: Add 5 elements to the linked list
    list.add(10); // Add 10 to the list
    list.add(20); // Add 20 to the list
    list.add(30); // Add 30 to the list
    list.add(40); // Add 40 to the list
    list.add(50); // Add 50 to the list
}
```

```

public static void main(String[] args) {
    // Step 6: Create a new LinkedList object
    LinkedList list = new LinkedList();

    // Step 7: Add 5 elements to the linked list
    list.add(10); // Add 10 to the list
    list.add(20); // Add 20 to the list
    list.add(30); // Add 30 to the list
    list.add(40); // Add 40 to the list
    list.add(50); // Add 50 to the list

    // Print the list before replacing any element
    System.out.println("Linked List before replacing 3rd element:");
    list.printList(); // Expected output: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

    // Step 8: Replace the 3rd element (index 2) with a new value (e.g., 99)
    list.replaceElementAt(index: 2, newValue: 99); // Replace the element at index 2 with 99

    // Print the list after replacing the 3rd element
    System.out.println("Linked List after replacing 3rd element:");
    list.printList(); // Expected output: 10 -> 20 -> 99 -> 40 -> 50 -> NULL
}

```

```

Linked List before replacing 3rd element:
10 -> 20 -> 30 -> 40 -> 50 -> NULL
Linked List after replacing 3rd element:
10 -> 20 -> 99 -> 40 -> 50 -> NULL

Process finished with exit code 0

```

Task 6:

Wap to create a linked list to add 5 elements and remove any element and display

```
public class LinkedListDemo {

    // Node class to represent each element in the linked list
    static class Node { 6 usages
        int data;    // Data of the node 2 usages
        Node next;  // Reference to the next node 6 usages

        // Constructor to initialize node
        public Node(int data) { 1 usage
            this.data = data;
            this.next = null; // Initially, no next node
        }
    }

    // LinkedList class to manage nodes
    static class LinkedList { 2 usages
        Node head; // Head of the linked list 7 usages
        int size;  // Size of the linked list 4 usages

        // Constructor to initialize an empty list
        public LinkedList() { 1 usage
            head = null;
            size = 0;
        }
    }
}
```

```

// Check if the list is empty
public boolean isEmpty() { 3 usages
    return head == null;
}

// Method to add a new element at the end of the list
public void add(int value) { 5 usages
    Node newNode = new Node(value); // Create a new node with the given value
    if (isEmpty()) {
        head = newNode; // If list is empty, new node becomes the head
    } else {
        Node current = head;
        // Traverse the list to reach the last node
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode; // Link the new node to the last node
    }
    size++; // Increase the size of the list
}

// Method to remove the first element from the list
public void removeFirst() { 1 usage
    Activate

```

```

public void removeFirst() { 1 usage
    if (isEmpty()) {
        System.out.println("The list is empty! No element to remove.");
    } else {
        head = head.next; // Remove the first element by pointing head to t
        size--; // Decrease the size
    }
}

// Method to display all the elements of the list
public void display() { 2 usages
    if (isEmpty()) {
        System.out.println("The list is empty.");
    } else {
        Node current = head;
        System.out.print("Linked List: ");
        while (current != null) {
            System.out.print(current.data + " -> ");
            current = current.next;
        }
        System.out.println("NULL"); // Indicating the end of the list
    }
}

```

Activate W

71:1 C# to Java


```

    public void display() { no usages
    }

    // Method to get the size of the list
    public int size() { no usages
        return size;
    }

}

// Main method to execute the program
public static void main(String[] args) {
    LinkedList list = new LinkedList(); // Create a new linked list

    // Add 5 elements to the list
    list.add(10);
    list.add(20);
    list.add(30);
    list.add(40);
    list.add(50);

    // Import static method... jdk.internal.org.jline.keymap.KeyMap.display()? Alt+Enter
    System.out.println("Original list: ");
    list.display(); // Expected Output: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
}

```

```

// Add 5 elements to the list
list.add(10);
list.add(20);
list.add(30);
list.add(40);
list.add(50);

// Display the list after adding elements
System.out.println("Original list:");
s    list.display(); // Expected Output: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

// Remove the first element from the list
list.removeFirst();

// Display the list after removal of the first element
System.out.println("List after removing the first element:");
💡    list.display(); // Expected Output: 20 -> 30 -> 40 -> 50 -> NULL

// Display the size of the list
System.out.println("Size of the list: " + list.size()); // Output: 4
}
}

```

```

Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
List after removing the first element:
Linked List: 20 -> 30 -> 40 -> 50 -> NULL
Size of the list: 4

Process finished with exit code 0

```

Task 7:

Wap to create a linked list to add 5 elements and display the list using for (use get()) and for each loops

```
public class LinkedListExample {  
  
    // Define the Node class  
    static class Node { 8 usages  
        int data; // Stores the value of the node 3 usages  
        Node next; // Points to the next node in the list 6 usages  
  
        public Node(int data) { 1 usage  
            this.data = data;  
            this.next = null; // Initially, the next pointer is null  
        }  
    }  
  
    // Define the LinkedList class  
    static class LinkedList { 2 usages  
        Node head; // Head of the list 5 usages
```

```
        static class LinkedList { 2 usages  
            Node head; // Head of the list 5 usages  
            int size; // Size of the list 4 usages  
  
            // Check if the list is empty  
            public boolean isEmpty() { 3 usages  
                return head == null;  
            }  
  
            // Get the size of the list  
            public int size() { no usages  
                return size;  
            }  
        }
```

```
        // Add element at the end of the list
```

```
public void add(int value) { 5 usages
    Node newNode = new Node(value);
    if (isEmpty()) {
        head = newNode; // If the list is empty, make the new node the
    } else {
        Node current = head;
        // Traverse to the last node
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode; // Link the new node to the last node
    }
    size++; // Increase the size of the list
}

// Get element at a specific index
public int get(int index) { 1 usage
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("Index out of range");
    }
    Node current = head;
    for (int i = 0; i < index; i++) {
        current = current.next;
    }
}
```

```
        current = current.next,  
    }  
    return current.data;  
}  
  
// Display the list using for loop (using get())  
public void displayUsingForLoop() { 1 usage  
    if (isEmpty()) {  
        System.out.println("The list is empty.");  
        return;  
    }  
    System.out.print("List using for loop (get()): ");  
    for (int i = 0; i < size; i++) {  
        System.out.print(get(i) + " -> ");  
    }  
    System.out.println("NULL");  
}  
  
// Display the list using for-each loop  
public void displayUsingForEachLoop() { 1 usage  
    if (isEmpty()) {  
        System.out.println("The list is empty.");  
        return;  
    }  
}
```

Activate Windows

```

        return;
    }
    System.out.print("List using for-each loop: ");
    Node current = head;
    for (Node node = current; node != null; node = node.next) {
        System.out.print(node.data + " -> ");
    }
    System.out.println("NULL");
}

// Main method to test the LinkedList
public static void main(String[] args) {
    LinkedList list = new LinkedList(); // Create a new linked list

    // Add 5 elements to the linked list
    list.add(10);
    list.add(20);
    list.add(30);
    list.add(40);
    list.add(50);
}

```

```
86 public static void main(String[] args) {
87     // Add 5 elements to the linked list
88     list.add(10);
89     list.add(20);
90     list.add(30);
91     list.add(40);
92     list.add(50);
93
94     // Display the linked list using the 'for' loop
95     list.displayUsingForLoop(); // Output: List
96
97     // Display the linked list using the 'for-each' loop
98     list.displayUsingForEachLoop(); // Output: List
99
100 }
101
102 }
```

LinkedListExample x

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editi
List using for loop (get()): 10 -> 20 -> 30 -> 40 -> 50 -> NULL
List using for-each loop: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

Process finished with exit code 0
```

Task 8:

Create a linked list and few items and convert it into an array

```

public class LinkedListToArrayExample {

    // Node class to represent each element of the list
    static class Node { 7 usages
        int data;    // Stores the data for each node 3 usages
        Node next;  // Points to the next node in the list 6 usages

        public Node(int data) { 1 usage
            this.data = data;
            this.next = null;
        }
    }

    // LinkedList class to manage the nodes
    static class LinkedList { 2 usages
        Node head;    // Head of the list 6 usages
        int size;     // Size of the list 2 usages

        // Add element to the end of the list
        public void add(int value) { 5 usages
            Node newNode = new Node(value);
            if (head == null) {
                head = newNode; // If the list is empty, set head
            } else {
                Node current = head;
                // Traverse to the last node
                while (current.next != null) {

```



```

        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode; // Link the new node
    }
    size++; // Increment the size of the list
}

// Convert the linked list to an array
public int[] toArray() { // 1 usage
    int[] array = new int[size]; // Create an array of the same size as the
    Node current = head;
    int index = 0;
    // Traverse the linked list and copy each element to the array
    while (current != null) {
        array[index++] = current.data;
        current = current.next;
    }
    return array; // Return the array
}

// Display the linked list
public void display() { // 1 usage
    if (head == null) {

```

```
public void display() { 1 usage
    if (head == null) {
        System.out.println("The list is empty.");
        return;
    }
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("NULL");
}
```

```
public static void main(String[] args) {
    LinkedList list = new LinkedList(); // Create a new linked list

    // Add some elements to the linked list
    list.add(10);
    list.add(20);
    list.add(30);
    list.add(40);
    list.add(50);|
```

A

71:22 66

```

list.add(10);
list.add(20);
list.add(30);
list.add(40);
list.add(50);

// Display the linked list
System.out.print("Linked List: ");
list.display(); // Output: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

// Convert the linked list to an array
int[] array = list.toArray();

// Display the array
System.out.print("Converted Array: ");
for (int value : array) {
    System.out.print(value + " ");
}
System.out.println(); // Output: Converted Array: 10 20 30 40 50
}
}

```

88:1 C8kFtc

```

C:\Program Files\Java\jdk-17\bin\java.exe
Linked List: 10 -> 20 -> 30 -> 40 -> 50
Converted Array: 10 20 30 40 50

Process finished with exit code 0

```

Task 9:

Create a linked list add few items and clone the 1st linked list with the 2nd linked list

```
public class LinkedListCloneExample {  
  
    // Node class to represent each element in the linked list  
    static class Node { 7 usages  
        int data; 3 usages  
        Node next; 6 usages  
  
        // Constructor to create a new node  
        public Node(int data) { 1 usage  
            this.data = data;  
            this.next = null;  
        }  
    }  
  
    // LinkedList class to manage the linked list operations  
    static class LinkedList { 6 usages  
        Node head; 5 usages  
        int size; 1 usage  
  
        // Method to check if the list is empty  
        public boolean isEmpty() { 2 usages  
            return head == null;  
        }  
  
        // Method to add an element to the linked list  
        public void add(int value) { 6 usages  
            Node newNode = new Node(value);
```

```

public void add(int value) { 6 usages
    Node newNode = new Node(value);
    if (isEmpty()) {
        head = newNode;
    } else {
        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    size++;
}

// Method to clone the current linked list into a new one
public LinkedList cloneList() { 1 usage
    LinkedList clonedList = new LinkedList();
    Node current = head;
    while (current != null) {
        clonedList.add(current.data); // Add each element from the original list
        current = current.next;
    }
    return clonedList;
}

```

Activat

87:1 C# to 8

```

        return clonedList;
    }

    // Method to display the elements of the linked list
    public void display() { 2 usages
        if (isEmpty()) {
            System.out.println("The list is empty.");
            return;
        }
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " -> ");
            current = current.next;
        }
        System.out.println("NULL");
    }
}

public static void main(String[] args) {
    // Create the first linked list and add some elements
    LinkedList list1 = new LinkedList();
    list1.add(10);
    list1.add(20);
}

```

```

public static void main(String[] args) {
    // Create the first linked list and add some elements
    LinkedList list1 = new LinkedList();
    list1.add(10);
    list1.add(20);
    list1.add(30);
    list1.add(40);
    list1.add(50);

    // Display the first linked list
    System.out.print("Original Linked List (list1): ");
    list1.display();

    // Clone the first list to a second list
    LinkedList list2 = list1.cloneList();

    // Display the cloned linked list
    System.out.print("Cloned Linked List (list2): ");
    list2.display();
}
}

```

```

Original Linked List (list1): 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Cloned Linked List (list2): 10 -> 20 -> 30 -> 40 -> 50 -> NULL

Process finished with exit code 0

```

Task 10:

Create linked list and iterate the values using ListIterator class in util package

```

import java.util.LinkedList;
import java.util.ListIterator;
💡
> public class LinkedListUsingListIterator {
>
    public static void main(String[] args) {
        // Create a LinkedList and add some elements
        LinkedList<String> list = new LinkedList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Orange");
        list.add("Mango");

        // Create a ListIterator for the LinkedList
        ListIterator<String> iterator = list.listIterator();

        // Iterate through the LinkedList using ListIterator
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
    }
}

```

```

↑ Apple
↓ Banana
= Orange
↔ Mango
=↓

```

Tsk 11:
Create a linked list and use push and pop methods.


```

import java.util.LinkedList;

> public class SimpleLinkedList {
>     public static void main(String[] args) {
        // Create a LinkedList of Strings
        LinkedList<String> list = new LinkedList<>();

        // Push elements to the list (add to the front of the list)
        list.push(e: "Apple");
        list.push(e: "Banana");
        list.push(e: "Orange");

        // Display the list after push
        System.out.println("List after push operations: " + list);

        // Pop elements from the list (remove from the front of the list)
        String removedElement = list.pop();
        System.out.println("Removed element: " + removedElement);

        // Display the list after pop operation
        System.out.println("List after pop operation: " + list);
    }
}

```

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\
List after push operations: [Orange, Banana, Apple]
Removed element: Orange
List after pop operation: [Banana, Apple]

Process finished with exit code 0

> src > SimpleLinkedList > main

```

Task 12:

Difference between Iterator and spliterator

Iterator

- Used to traverse elements in a sequential manner (one-by-one).
 - Supports operations like `hasNext()`, `next()`, and `remove()`.
 - Works with Collections like `List`, `Set`, etc.
 - Cannot be split or processed in parallel.
 - Introduced in Java 1.2.
-

Splititerator

- Short for "Splitable Iterator".
- Supports parallel processing by dividing elements into multiple parts.
- Has methods like `tryAdvance()`, `trySplit()`, and `forEachRemaining()`.
- Designed for use with Streams and parallelism.
- Introduced in Java 8.

Iterator:

An Iterator is used to sequentially traverse elements in a collection one by one. It provides methods like `hasNext()`, `next()`, and `remove()` for simple, forward-only iteration.

Splititerator:

A Splititerator (splitable iterator) is designed for parallel traversal by splitting a collection into multiple parts. It supports both sequential and parallel iteration, mainly used with the Streams API.

Iterator:

An Iterator is used to sequentially traverse elements in a collection one by one. It provides methods like `hasNext()`, `next()`, and `remove()` for simple, forward-only iteration.

Splititerator:

A Splititerator (splitable iterator) is designed for parallel traversal by splitting a collection into multiple parts. It supports both sequential and parallel iteration, mainly used with the Streams API.

Task 13:

Below is the code for Split iterator... run it to see the output..

Can you it to sout()... and see ..

16.38 to 16.43

```
import java.util.*;
```

```
public class Task0013_DS_Linkedlist_Splitlterator {  
    public static void main(String[] args) {  
  
        LinkedList<String> lobj = new LinkedList<>();  
  
        lobj.add("Prasunamba");  
        lobj.add("Meher");  
        lobj.add(".MK");  
  
        Spliterator<String> sitobj = lobj.spliterator();  
  
        System.out.println("Splitting the list:");  
        sitobj.forEachRemaining(System.out::println);  
    }  
}
```

```

1  import java.util.*;
2
3  public class Task0013_DS_Linkedlist_SplitIterator {
4      public static void main(String[] args) {
5
6          LinkedList<String> lobj = new LinkedList<>();
7
8          lobj.add("Meera");
9          lobj.add("j");
10         lobj.add(".MK");
11
12         Spliterator<String> sitobj = lobj.spliterator();
13
14         System.out.println("Splitting the list:");
15         sitobj.forEachRemaining(System.out::println);
16     }
17 }

```

```

↑ -----
↓ Meera
↓ j
↓ .MK
↓
↓ Process finished with exit co

```

```

main.java Task0013_DS_Linkedlist_SplitIterator.java
import java.util.*;

> public class Task0013_DS_Linkedlist_SplitIterator {
>     public static void main(String[] args) {

        // Creating a LinkedList of Strings
        LinkedList<String> lobj = new LinkedList<>();

        // Adding elements to the LinkedList
        lobj.add("Prasunamba");
        lobj.add("Meher");
        lobj.add(".MK");

        // Getting the Spliterator for the LinkedList
        Spliterator<String> sitobj = lobj.spliterator();

        // Printing a message to indicate the splitting of the list
        System.out.println("Splitting the list:");

        // Using forEachRemaining to print each element of the list using Syst
        sitobj.forEachRemaining( String item -> System.out.println(item));
    }
}

```

```

Prasunamba
Meher
.MK

Process finished with exit code 0

```

Task 14:

Create a linkedlist and display items into 2 lists using split iterator

```

import java.util.*;

public class Task14_LinkedList_SplitIterator {
    public static void main(String[] args) {

        // Create a LinkedList and add some elements
        LinkedList<String> lobj = new LinkedList<>();
        lobj.add("Meera");
        lobj.add("j");
        lobj.add(".MK");
        lobj.add("Hello");
        lobj.add("World");

        // Create a Spliterator from the LinkedList
        Spliterator<String> sitobj = lobj.spliterator();

        // Create a new list to store the first half of the elements
        List<String> list1 = new ArrayList<>();
        // Create a new list to store the second half of the elements
        List<String> list2 = new ArrayList<>();

        // Split the list into two
        // We will first try to split the original Spliterator into two
        Spliterator<String> split = sitobj.trySplit();

        // Add elements from the first Spliterator into list1
        if (split != null) {
            // Add elements from the first Spliterator into list1
        }
    }
}

```

Activate

29:10 C# To Java

```

// Add elements from the first Splitter into list1
if (split != null) {
    split.forEachRemaining(list1::add);
}

// Add the remaining elements into list2
sitobj.forEachRemaining(list2::add);

// Display the two lists
System.out.println("First List: " + list1);
System.out.println("Second List: " + list2);
}
}

```

```

C:\Program Files\Java\jdk-17\bin>java.exe
First List: [Meera, j, .MK, Hello, World]
Second List: []

Process finished with exit code 0

```

Task 14:

Create a linked list and display items into 2 lists using split iterator

```
public class Task14_LinkedList_SplitIterator {
    public static void main(String[] args) {

        // Creating a linked list and adding some elements
        LinkedList<String> lobj = new LinkedList<>();
        lobj.add("Meera");
        lobj.add("j");
        lobj.add(".MK");
        lobj.add("Hello");
        lobj.add("World");

        // Getting the Spliterator object
        Spliterator<String> spliterator = lobj.spliterator();

        // Create two lists for storing the split elements
        LinkedList<String> list1 = new LinkedList<>();
        LinkedList<String> list2 = new LinkedList<>();

        // Use the original spliterator to add elements to list1
        int splitIndex = 2; // Decide where to split (after 2 elements)

        for (int i = 0; i < splitIndex && spliterator.tryAdvance(list1::add); i++) {
            // Adding elements to the first list until the split point
        }

        // The rest of the elements should go into the second list
        spliterator.forEachRemaining(list2::add);
    }
}
```




```
}

// The rest of the elements should go into the second list
spliterator.forEachRemaining(list2::add);

// Displaying the results
System.out.println("First List: " + list1);
System.out.println("Second List: " + list2);
}
```

```
"C:\Program Files\Java\jdk-17\bin\jav
First List: [Meera, j]
Second List: [.MK, Hello, World]

Process finished with exit code 0
```

```

R Rename usages
public class SplitLinkedListExample{
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>(Arrays.asList(
            "Meera", "J", ".MK", "Hello", "World", "Java", "GPT", "Code", "Data"
        ));

        int halfSize = list.size() / 2;

        LinkedList<String> firstHalf = new LinkedList<>(list.subList(0, halfSize));
        LinkedList<String> secondHalf = new LinkedList<>(list.subList(halfSize, list.size()));

        System.out.println("First half of the list:");
        for (String s : firstHalf) {
            System.out.println(s);
        }

        System.out.println("Second half of the list:");
        for (String s : secondHalf) {
            System.out.println(s);
        }
    }
}

```

```

First half of the list:
Meera
J
.MK
Hello
Second half of the list:

```

```

World
Java
GPT
Code
Data

```

Task 15:

What do you understand by a pointer?

A **pointer** is a variable that **stores the memory address** of another variable. Instead of holding a value directly, it points to the location where the value is stored in memory.

Task 16:

Difference between * and & in pointers?

& (Address-of Operator):

- Used to **get the memory address** of a variable.
- It means "**address of**".

♦ Example: `int x = 5;`

`int* ptr = &x; // &x gives the address of variable x`

*** (Dereference Operator):**

- Used to **access the value at the address** a pointer is pointing to.
- It means "**value at**" or **dereference**.

♦ Example:

`int x = 5;`

`int* ptr = &x;`

`cout << *ptr; // prints 5, which is the value at the address stored in ptr`

& → gives the **address** of a variable.

***** → gives the **value** stored at a pointer's address.

Task 17:

Wap in c or c++ to implement the use of pointers.

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 int main() { 5 int num = 10; // Normal integer variable 6 int* ptr = &num; // Pointer storing address of 'num' 7 8 // Display the value and address 9 cout << "Value of num: " << num << endl; 10 cout << "Address of num (&num): " << &num << endl; 11 cout << "Value stored in ptr (address of num): " << ptr << endl; 12 cout << "Value pointed to by ptr (*ptr): " << *ptr << endl; 13 14 // Modify value using pointer 15 *ptr = 20; 16 17 cout << "\nAfter modifying value using pointer:" << endl; 18 cout << "New value of num: " << num << endl; 19 cout << "Value pointed to by ptr: " << *ptr << endl; 20 21 return 0; 22 }</pre>		Value of num: 10 Address of num (&num): 0x7ffdbbbf9684 Value stored in ptr (address of num): 0x7ffdbbbf9684 Value pointed to by ptr (*ptr): 10 After modifying value using pointer: New value of num: 20 Value pointed to by ptr: 20 === Code Execution Successful ===

Task 18:

Wap to create a doubly linked list

```

class DoublyLinkedList {
    // Node class inside DoublyLinkedList
    class Node { 6 usages
        int data; 3 usages
        Node prev, next; 6 usages

        Node(int data) { 1 usage
            this.data = data;
        }
    }

    private Node head, tail; 9 usages

    // Add element at the end
    public void add(int data) { 4 usages
        Node newNode = new Node(data);
        if (head == null) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    // Delete element by value (first occurrence)
    public void delete(int data) { 2 usages

```

```

// Delete element by value (first occurrence)
public void delete(int data) { 2 usages
    Node current = head;
    while (current != null) {
        if (current.data == data) {
            if (current == head) {
                head = head.next;
                if (head != null) head.prev = null;
            } else if (current == tail) {
                tail = tail.prev;
                if (tail != null) tail.next = null;
            } else {
                current.prev.next = current.next;
                current.next.prev = current.prev;
            }
            System.out.println("Deleted: " + data);
            return;
        }
        current = current.next;
    }
    System.out.println("Element not found: " + data);
}

```

```

// Traverse and print the list
public void traverse() { 3 usages

```

```
public void traverse() { 3 usages
    Node current = head;
    System.out.print("List: ");
    while (current != null) {
        System.out.print(current.data + " ");
        current = current.next;
    }
    System.out.println();
}

// Main method to test
public static void main(String[] args) {
    DoublyLinkedList list = new DoublyLinkedList();

    list.add(10);
    list.add(20);
    list.add(30);
    list.add(40);

    list.traverse(); // Output: 10 20 30 40

    list.delete(data: 20);
    list.traverse(); // Output: 10 30 40

    list.delete(data: 10);
```

